Data Structures and Algorithms - 2025 - 2026

Quiz navigation

1 ✓ 2 ✓ 3 ✗ 4 ✓ 5 ✓
6 ✓ 7 ✓ 8 ✓ 9 ✗ 10 ✓
11 ✓ 12 ✓

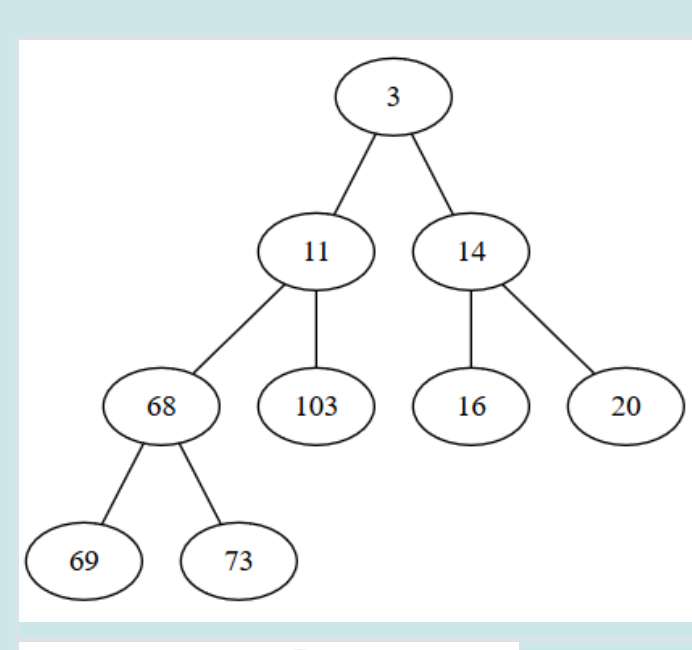
Show one page at a time

Finish review

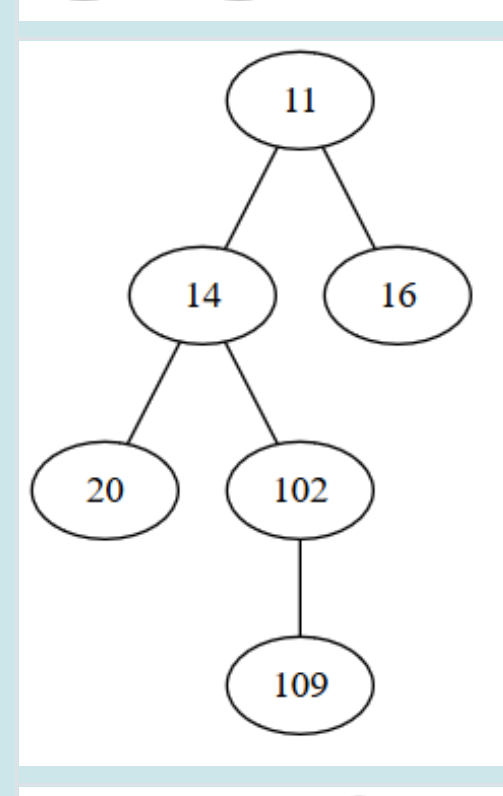
Status	Finished
Started	Sunday, 17 May 2026, 2:42 PM
Completed	Sunday, 17 May 2026, 3:08 PM
Duration	26 mins 17 secs
Marks	10.00/12.00
Grade	**8.33** out of 10.00 (**83.33**%)

Question **1**
Correct
Mark 1.00 out of 1.00
Flag question

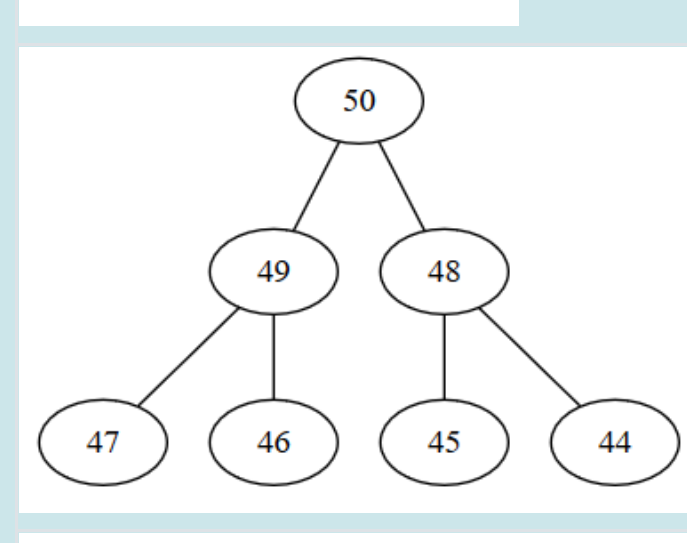
Match the drawings to the description.



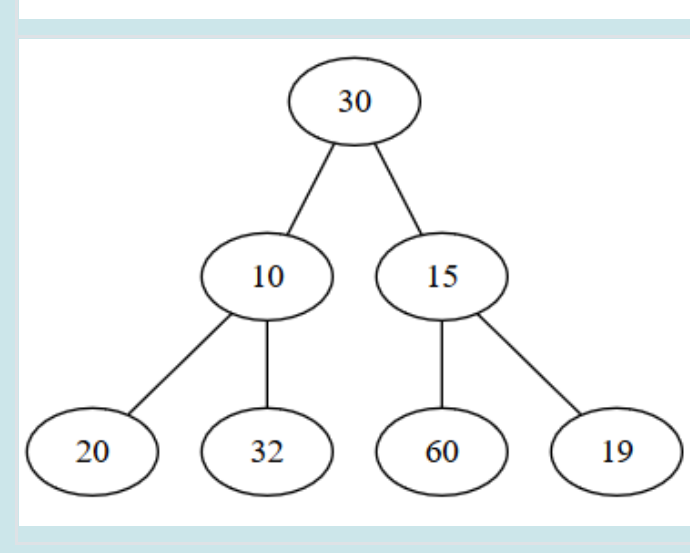
It is a valid min-heap. ✓



It does not have a heap structure, but it respects the heap property. ✓



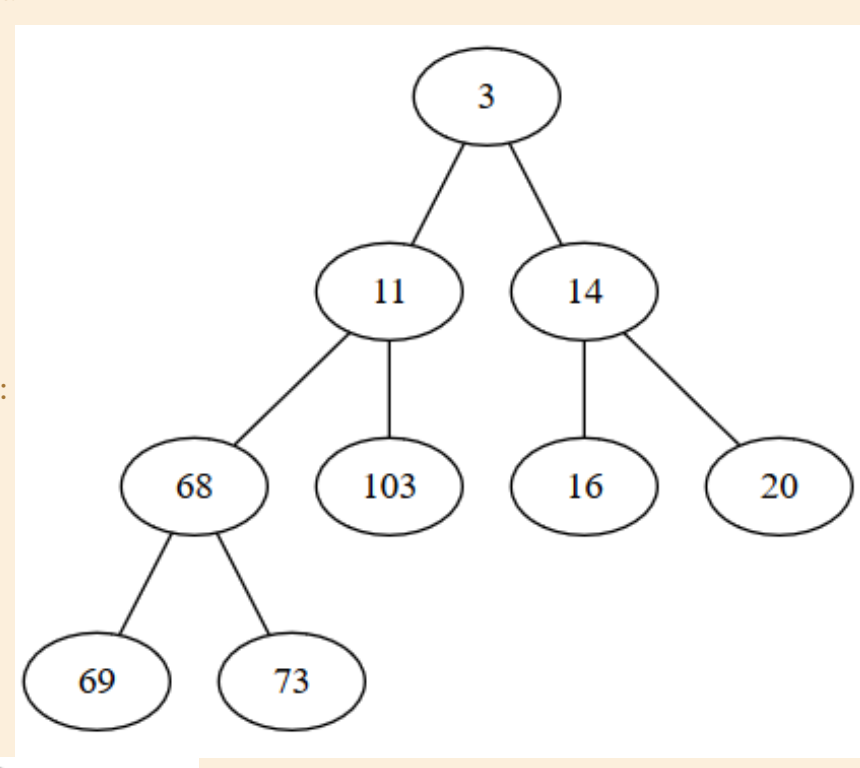
It is a valid max-heap. ✓



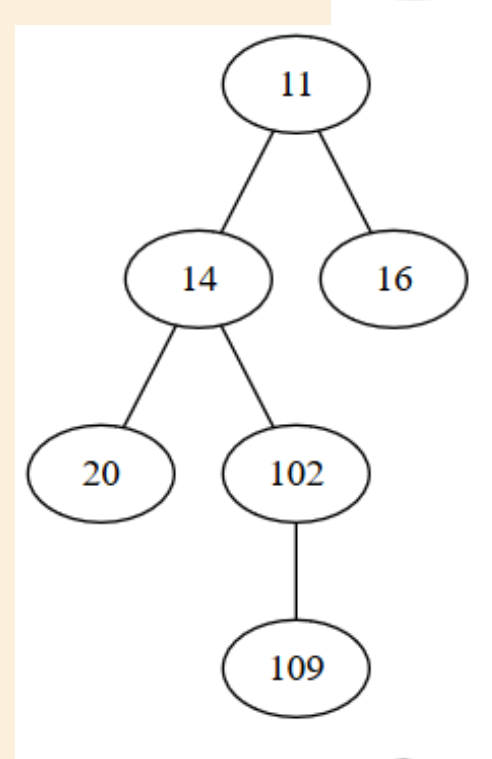
It does have a heap structure, but it does not respect the heap property ✓

Your answer is correct.

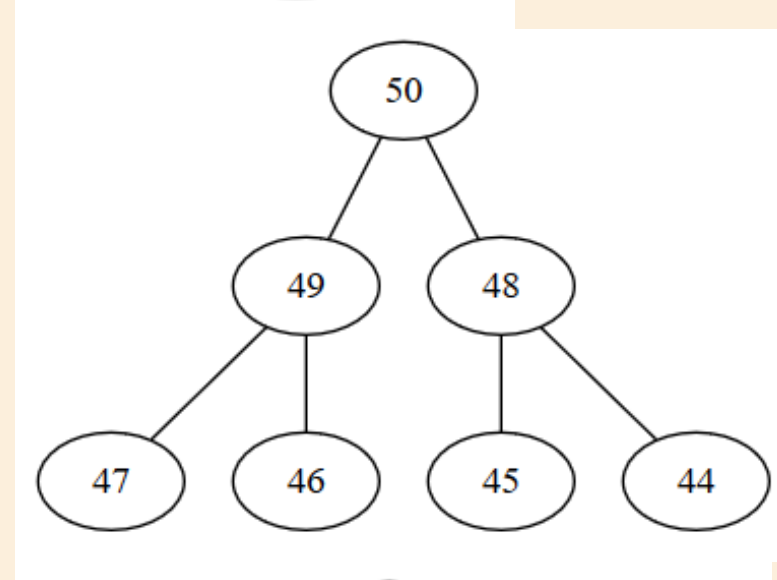
The correct answer is:



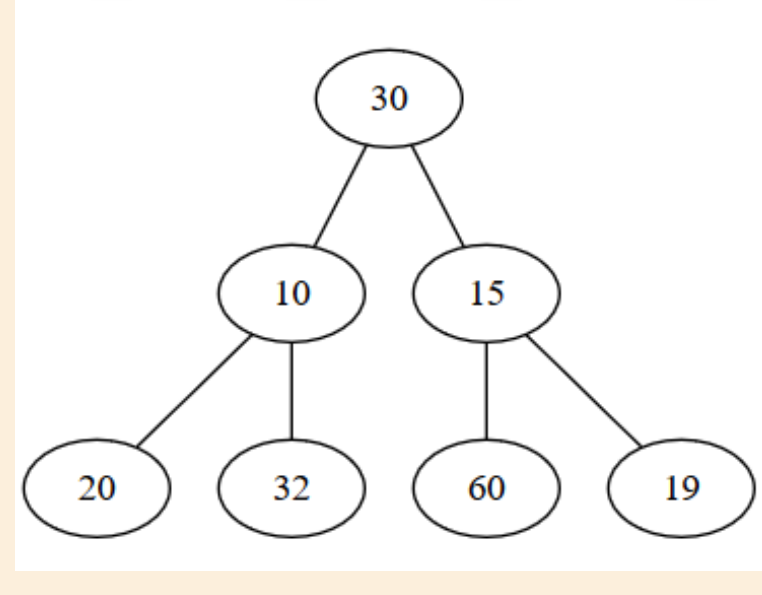
→ It is a valid min-heap.,



→ It does not have a heap structure, but it respects the heap property.,



→ It is a valid max-heap.,



→ It does have a heap structure, but it does not respect the heap property

Question **2**
Correct
Mark 1.00 out of 1.00
Flag question

Assume you have the following min-heap.

[19, 60, 20, 80, 101, 25, 40, 99, 103]

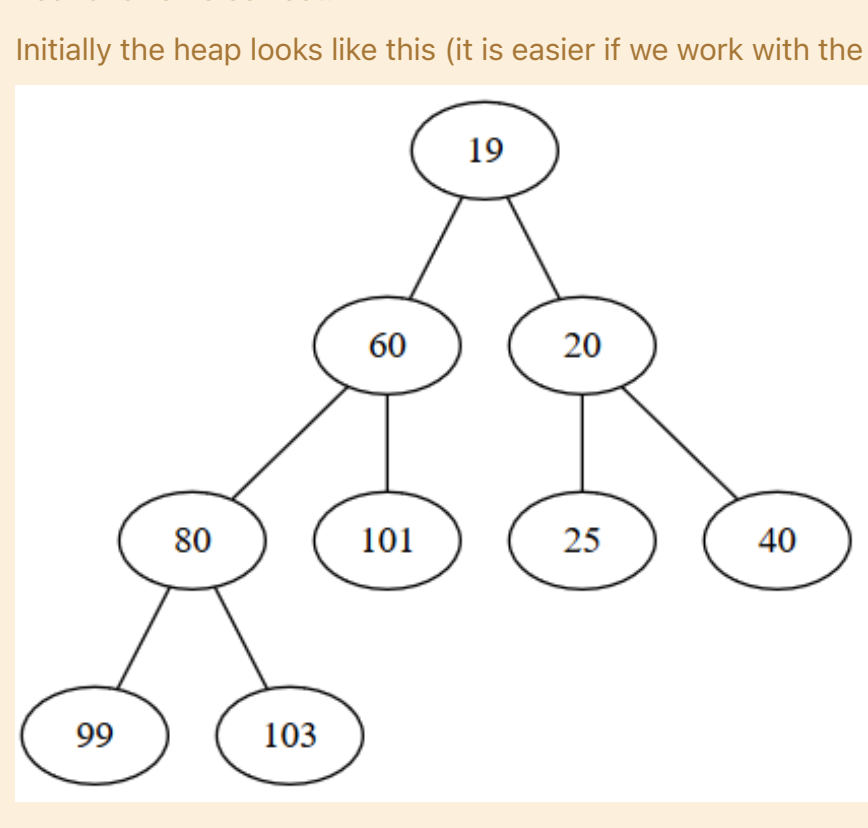
How is the heap going to look like after adding the element 23?

Select one or more:

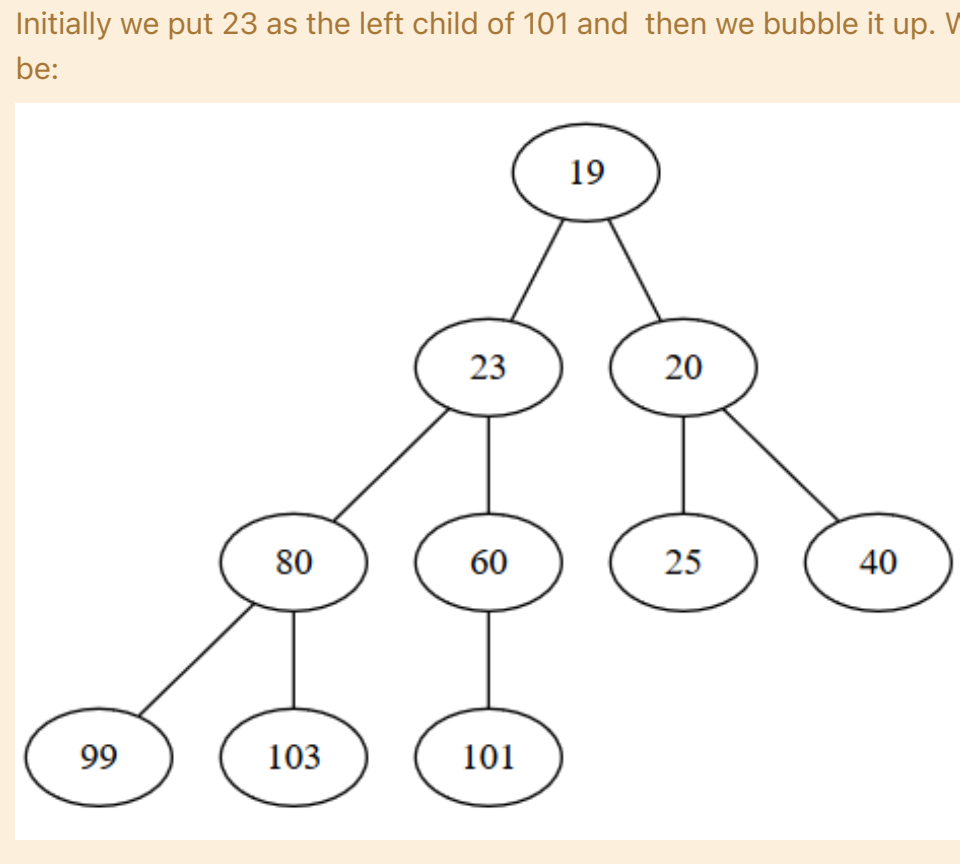
- ☐ [19, 60, 20, 80, 101, 25, 40, 99, 103, 23]
- ☐ [19, 60, 20, 23, 80, 101, 25, 40, 99, 103]
- ☑ [19, 23 20, 80, 60, 25, 40, 99, 103, 101] ✓
- ☐ [19, 60, 20, 80, 101, 25, 40, 99, 23, 103]
- ☐ [19, 20, 23, 80, 60, 101, 25, 40, 99, 103]

Your answer is correct.

Initially the heap looks like this (it is easier if we work with the tree representation):



Initially we put 23 as the left child of 101 and then we bubble it up. We will need to swap it with 101 and 60, so the final result will be:



The correct answer is: [19, 23 20, 80, 60, 25, 40, 99, 103, 101]

Question **3**
Incorrect
Mark 0.00 out of 1.00
Flag question

Assume you have a max-heap. What is the complexity of finding the minimum element of the heap?

Select one or more:

- ☐ Θ(1)
- ☐ Θ(log2n)
- ☑ Θ(n) ✓
- ☐ O(n)
- ☑ Θ(n/2) ✗

Your answer is incorrect.

In a max-heap every node is greater than its children. So the minimum must not have a child, it has to be a leaf node. The last element of the heap is on position n. Its parent is on position n/2. This is the last node with a child. Nodes between positions n/2+1 and n are leaf nodes. Any of them can be the minimum. We need to check aproimately n/2 nodes. As a complexity this is still Θ(n) (2 is just a constant and we can ignore it)

The correct answer is: Θ(n)

Question **4**
Correct
Mark 1.00 out of 1.00
Flag question

Assume you have a max-heap with 19 elements. How many levels does the heap have?

Select one or more:

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4
- ☑ 5 ✓
- ☐ 6
- ☐ 7
- ☐ 8

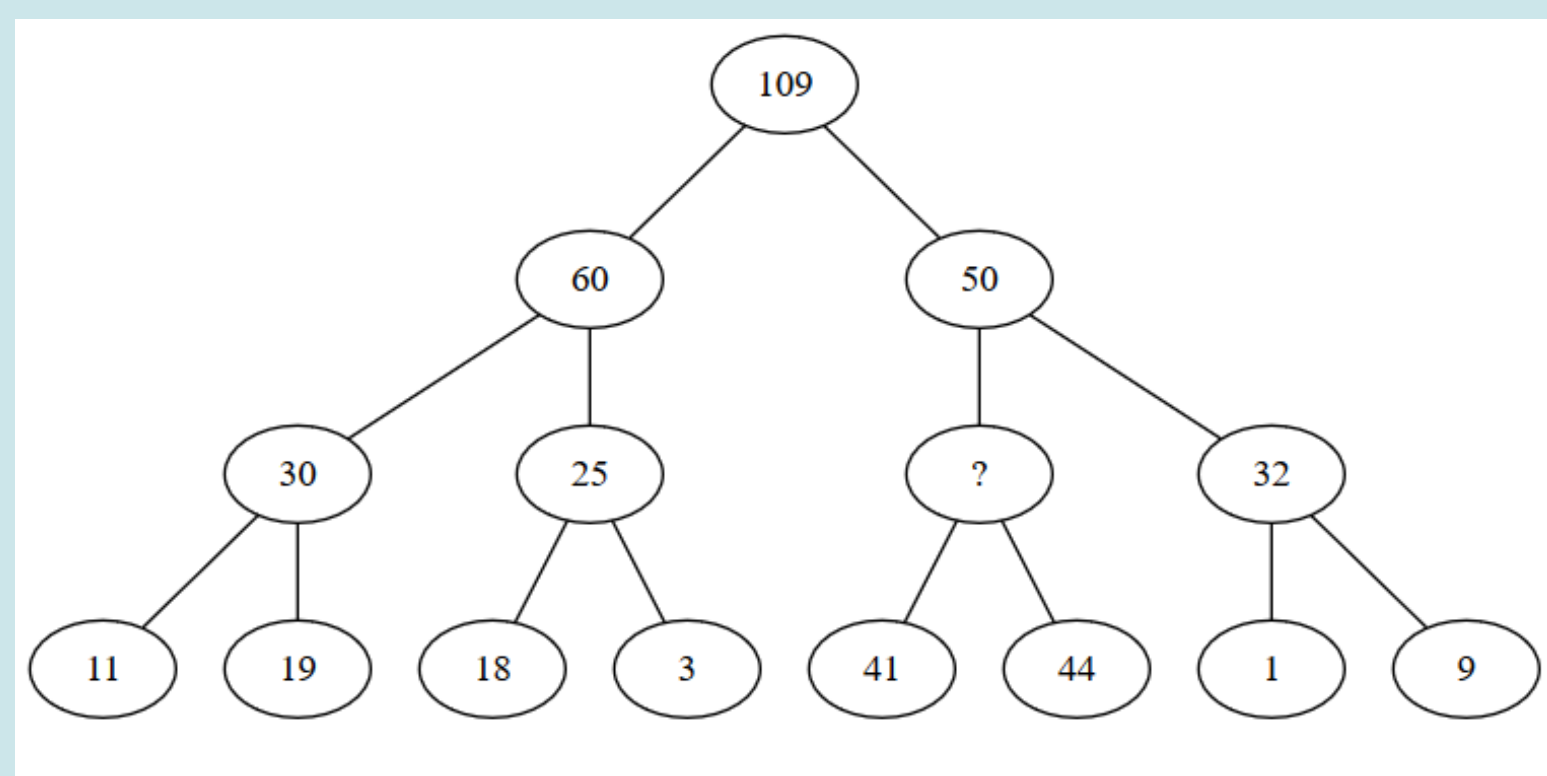
Your answer is correct.

We have the nodes on levels like this: 1 + 2 + 4 + 8 + 4 (last level is not complete). So 5 levels.

The correct answer is: 5

Question **5**
Correct
Mark 1.00 out of 1.00
Flag question

Consider the following binary heap that contains unique values:



Which one(s) of the following values can be in place of "?" ?

Select one or more:

- ☐ 30
- ☑ 47 ✓
- ☐ 40
- ☐ 41
- ☐ 52
- ☑ 49 ✓
- ☐ 10
- ☐ 35

Your answer is correct.

It is a max heap. Heap property tells us that every nodes has to be greater than its children. So ? has to be less than its parent (50) and greater than its children (41 and 44). Possible values are (considering unique elements): 45, 46, 47, 48, 49.

The correct answers are: 47, 49

Question **6**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that I have a max-heap. What is the complexity of finding the maximum element from the heap?

Select one or more:

- ☐ Θ(n)
- ☑ Θ(1) ✓
- ☐ Θ(log2 n)
- ☐ O(n)
- ☐ O(log2 n)

Your answer is correct.

In a max-heap the root is the maximum. It is on position 1.

The correct answer is: Θ(1)

Question **7**
Correct
Mark 1.00 out of 1.00
Flag question

In a max-heap what is the complexity of finding the second largest element?

Select one or more:

- ☑ Θ(1) ✓
- ☐ Θ(n)
- ☐ Θ(log2n)
- ☐ O(log2n)
- ☐ O(n)

Your answer is correct.

In a max-heap the root is the maximum. The second largest element has to be a child of the maximum, so it can only be on position 2 or 3. It means that we only need to check a constant number of positions.

The correct answer is: Θ(1)

Question **8**
Correct
Mark 1.00 out of 1.00
Flag question

Consider the following heap:

[19, 60, 20, 80, 101, 25, 40, 99, 103]

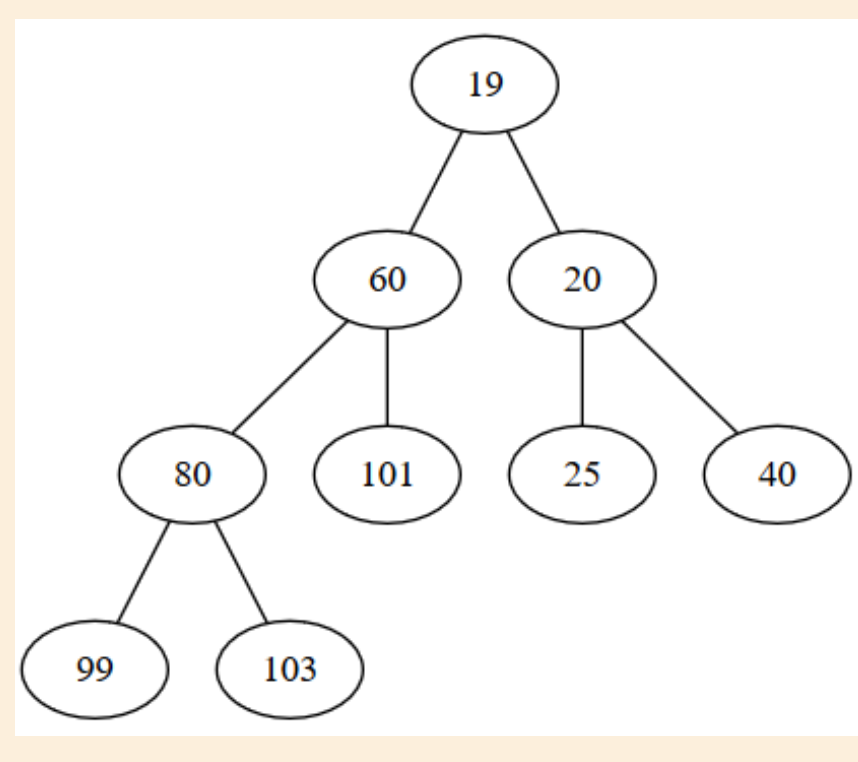
How will the heap look like if we remove an element?

Select one or more:

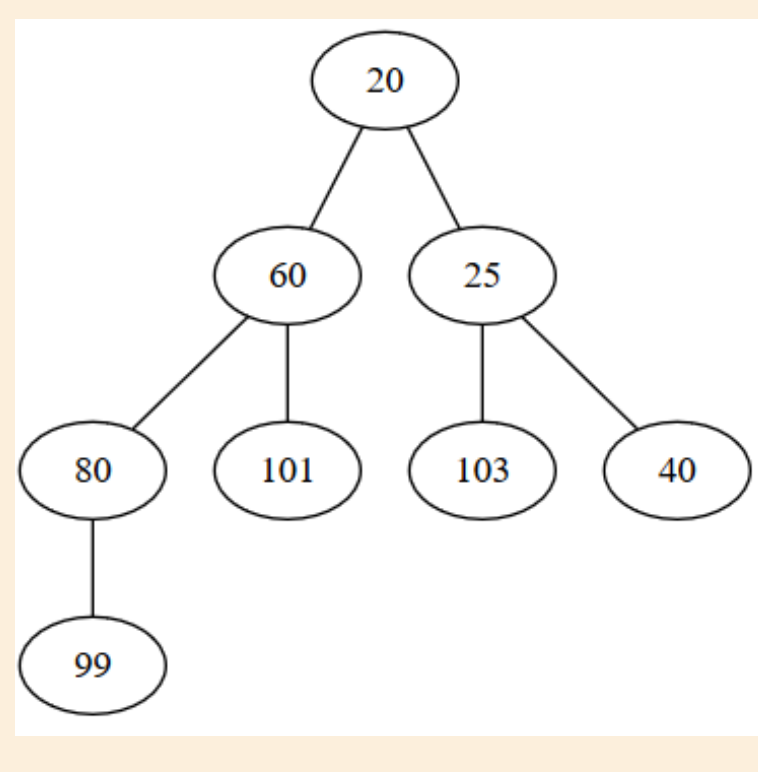
- ☑ [20, 60, 25, 80, 101, 103, 40, 99] ✓
- ☐ [19, 60, 20, 80, 101, 25, 40, 99]
- ☐ [103, 60, 20, 80, 101, 25, 40, 99]
- ☐ [20, 60, 25, 80, 101, 40, 99, 103]
- ☐ [20, 25, 60, 80, 101, 40, 99, 103]

Your answer is correct.

Initially the heap looks like this (it is easier to work with the tree format)



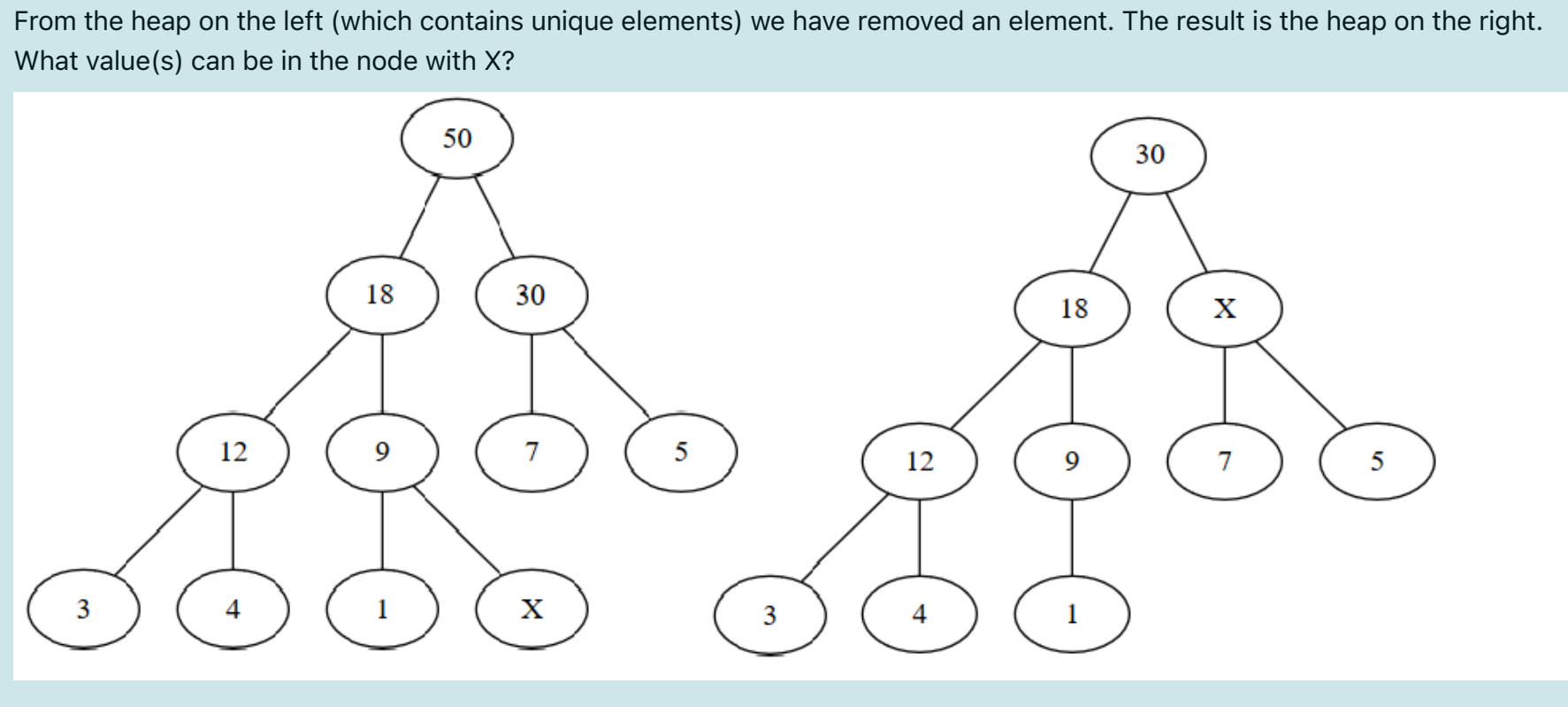
We can only remove the root. We move 103 in place of the root and bubble it down. It will change place with 20 and 25 (remember, we always compare it with the smallest child).



The correct answer is: [20, 60, 25, 80, 101, 103, 40, 99]

Question **9**
Incorrect
Mark 0.00 out of 1.00
Flag question

From the heap on the left (which contains unique elements) we have removed an element. The result is the heap on the right. What value(s) can be in the node with X?



Select one or more:

- ☐ 2
- ☑ 8 ✓
- ☑ 12 ✗
- ☑ 20 ✗
- ☐ 1

Your answer is incorrect.

It is a max-heap, every node is greater than its children. From the figure on the left we know that X is less than 9.

When we remove something, 50 is going to be removed and X will be moved to the root. Then we start bubble-down, we compare X with the maximum child, 30. We know from the result that X was swapped with 30, so 30 > X. Then, still in the bubble-down process, we compare X with the maximum of the new children, 7. Since X was not swapped with 7, we know that it must be greater than 7.

So we have:

X < 9

X > 7

X has to be 8.

The correct answer is: 8

Question **10**
Correct
Mark 1.00 out of 1.00
Flag question

Assume you have the following heap:

[13, 25, 19, 60, 28, 40, 50, 77, 80, 30, 29, 41]

How will the array look like after three delete operations?

Select one or more:

- ☐ 13 25 19 60 28 40 50 77 80
- ☐ 60 28 40 50 77 80 30 29 41
- ☑ 28 29 40 60 30 41 50 77 80 ✓
- ☐ 60 28 40 50 77 80 30 29 41
- ☐ 28 40 29 30 60 50 77 80 41

Your answer is correct.

Remember, you can only delete the root. You move the "last" element to replace the root and after that do a bubble-down.

The correct answer is: 28 29 40 60 30 41 50 77 80

Question **11**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that you have the array [100, 99, 98, 97, 96, 95, 94, 93, 92, 91] and we apply the *heapify* process on it to transform into a min-heap.

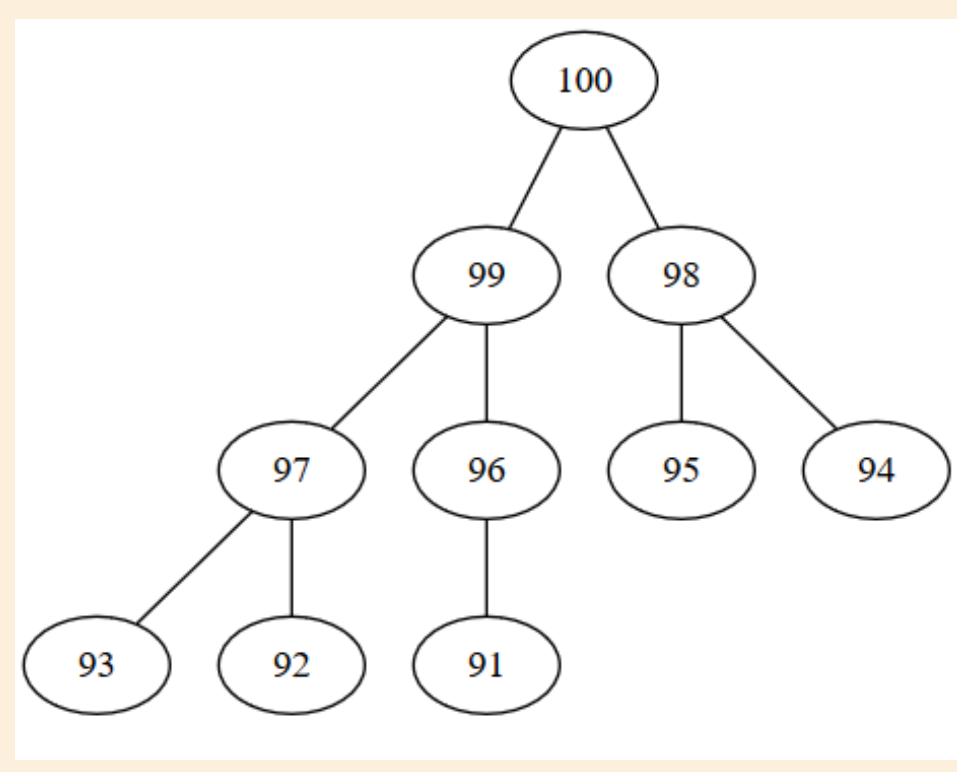
Obs: *Heapify* is an alternative possibility for transforming an entire array into a heap (instead of adding each element one after the other we call *bubble-down* on every element of the array, but starting from the end and going towards the first element).

Select one or more:

- ☑ [91, 92, 94, 93, 96, 95, 98, 100, 97, 99] ✓
- ☐ [91, 92, 93, 94, 95, 96, 97, 98, 99, 100]
- ☐ [91, 93, 92, 94, 95, 96, 97, 100, 98, 99]
- ☐ [91, 94, 92, 95, 96, 93, 98, 97, 99, 100]

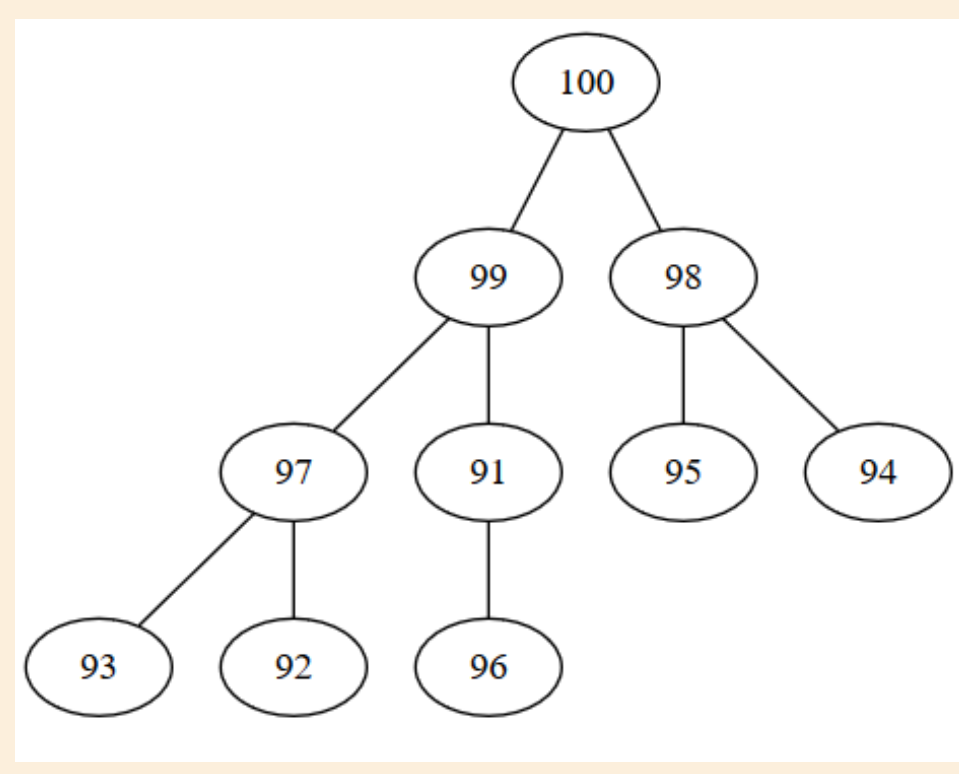
Your answer is correct.

Let's write the initial array as a tree.

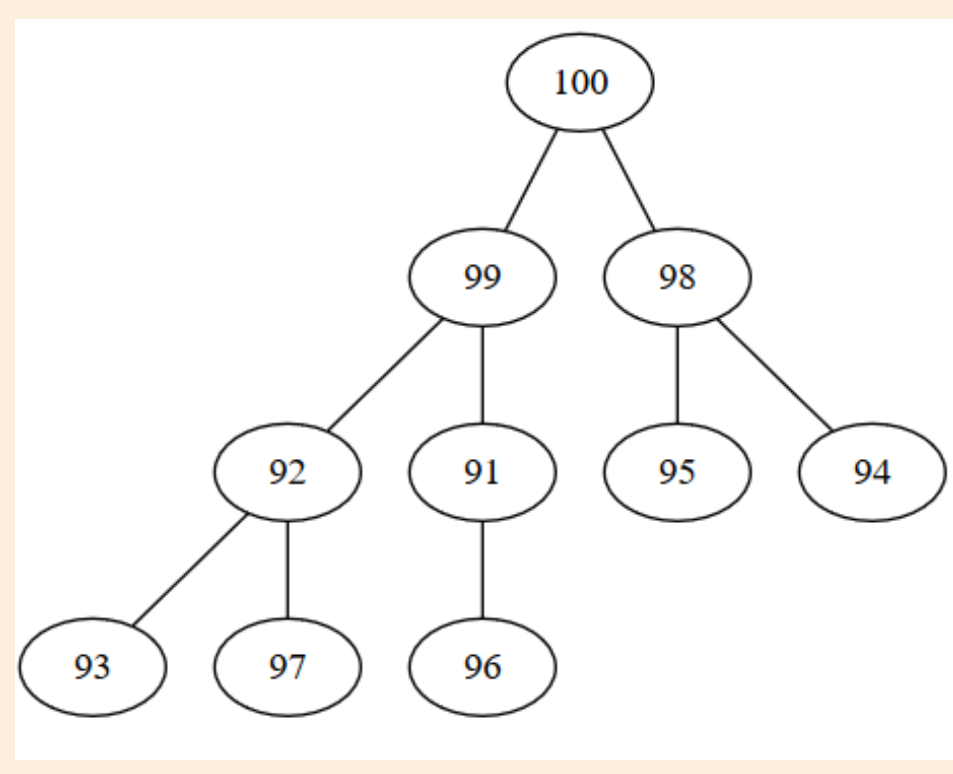


We start calling bubble down on the elements, starting from the end towards the beginning. When we call bubble down for 91, 92, 93, 94 ans 95 nothing happens (these nodes have no children).

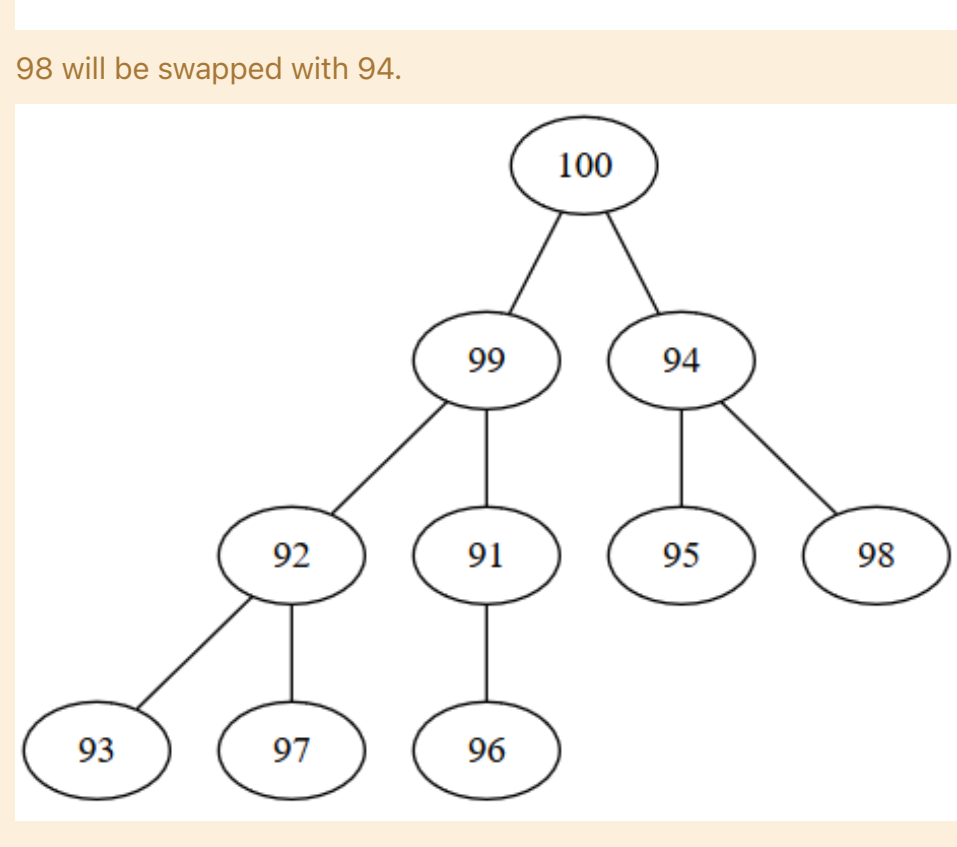
Then when we call bubble down for 96 it will be swapped with 91.



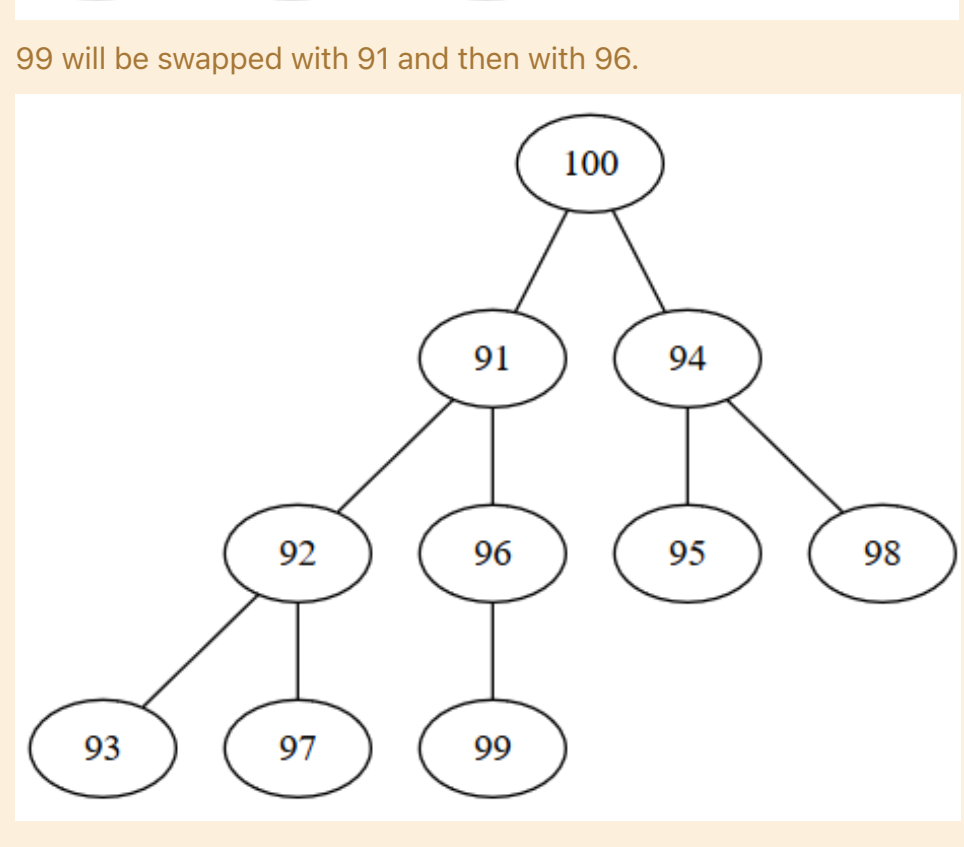
97 will be swapped with 92 (minimum of the two children)



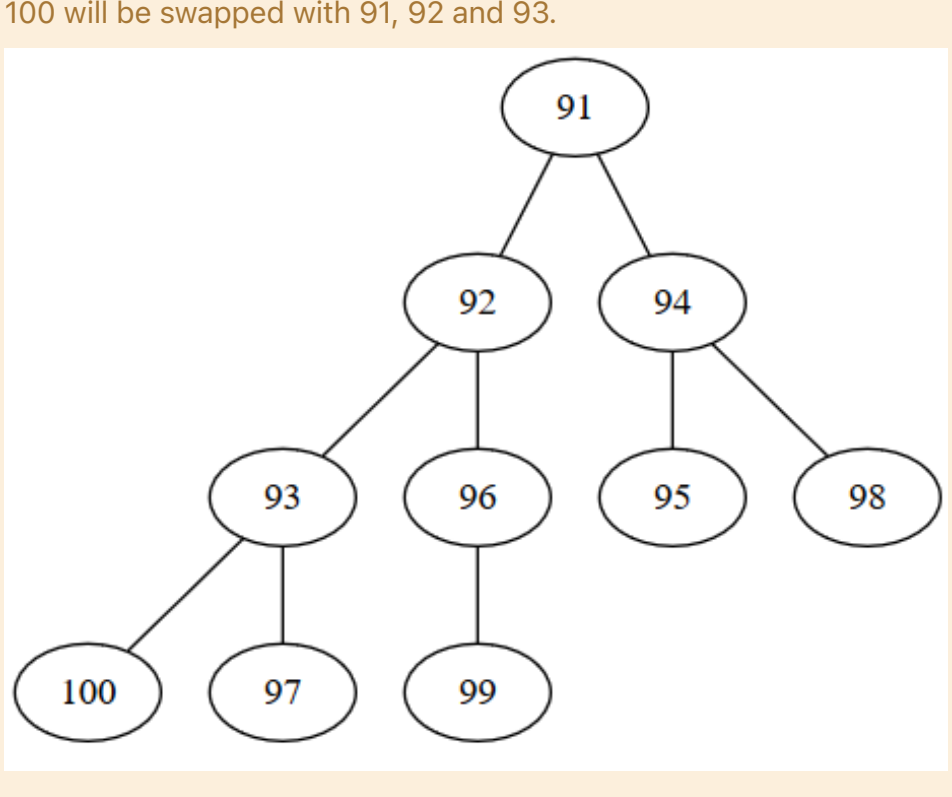
98 will be swapped with 94.



99 will be swapped with 91 and then with 96.



100 will be swapped with 91, 92 and 93.



The correct answer is: [91, 92, 94, 93, 96, 95, 98, 100, 97, 99]

Question **12**
Correct
Mark 1.00 out of 1.00
Flag question

Assume you want to sort in an ascending order an array of integer numbers, using HeapSort. If we do not want to use an extra array during the sorting, should we build a min-heap or a max-heap?

Select one or more:

- ☑ Max-heap ✓
- ☐ Min-heap
- ☐ Both of them can be used
- ☐ We cannot do HeapSort without an extra array, so none of them is suitable.

Your answer is correct.

If we want to do ascending sorting without an extra array, we need a max-heap. Max heap will have the maximum element as root (on the first position). When that element is removed, the elements of the heap will occupy one less position in the array, the last position being empty. We can put the maximum element (the removed one) there, because this is exactly where it should be in the sorted array.

In theory, we could use a min-heap as well, and build the solution in the reverse order (when you remove the minimum element, put it on the last position of the array, when you remove the new minimum, put it on the second position from the end, etc.) and then reverse the whole array. While this would not increase the complexity of heapsort, it is easier to build a max-heap.

The correct answer is: Max-heap

Finish review

◀ Lecture 7 Quiz Jump to... Lecture 9 Quiz ▶